	AIR UNIVERSITY
	DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
	EXPERIMENT NO 6

Lab Title: **Conversion infix to postfix and postfix to infix using Stack**

Student Name: **Muammad Waqas** **Reg. No:** **240756**

Objective: Design and learn algorithm to convert infix to postfix and prefix and the evaluate using . . stack concept in C++

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks: _____ Obtained Marks: _____

LAB REPORT ASSESSMENT:

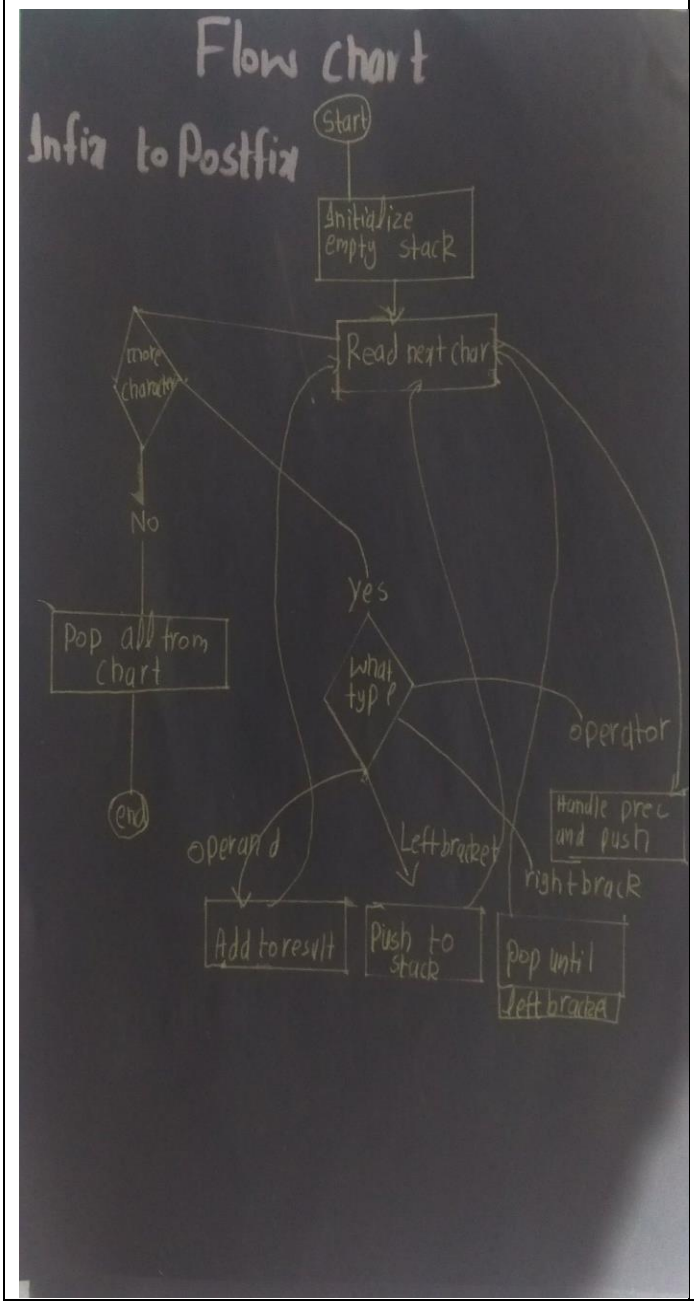
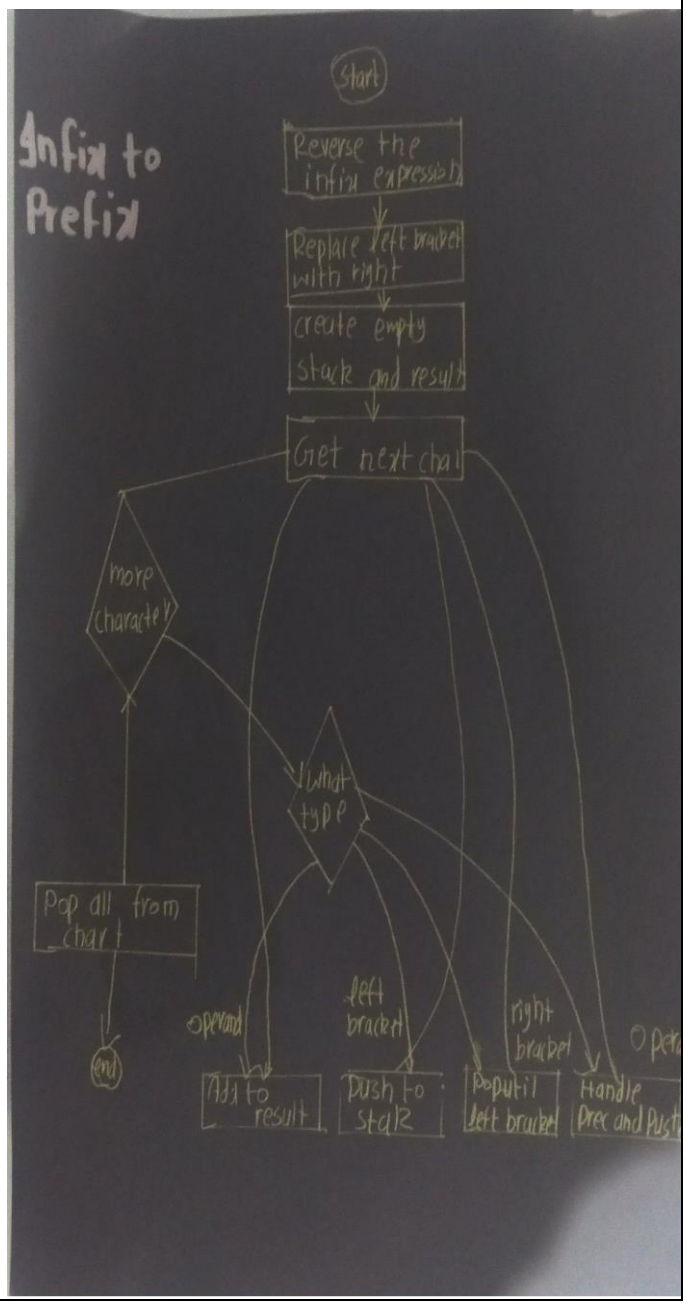
Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____

Date: _____ Signature: _____

OBJECTIVE:

1. To implement a C++ program that converts an infix expression into postfix and prefix forms using stack operations.
2. To understand and apply the concept of operator precedence and associativity in expression evaluation.
3. To demonstrate efficient use of stack data structure for managing operators and operands during expression conversions

INFIX TO POSTFIX FLOWCHART	INFIX TO PREFIX FLOWCHART
	

Lab Tasks :

1. Write code for the conversion of infix expression into postfix expression.

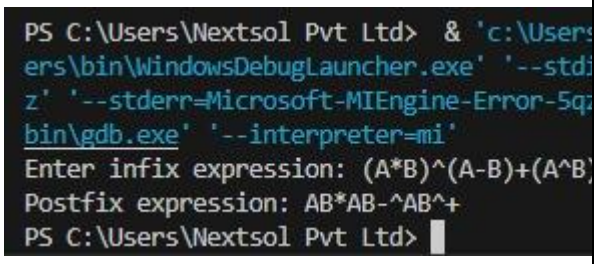
Code:

C: > Users > Nextsol Pvt Ltd > Desktop > C++ > mc

```
#include<iostream>
#include<string>
using namespace std;
struct Node{
char data;
Node*next;};
class Stack{
private:
Node*top;
public:
Stack(){
top=NULL;}
void push(char c){
Node*newNode=new Node();
newNode->data=c;
newNode->next=top;
top=newNode;}
char pop(){
if(top==NULL)return'\0';
char c=top->data;
Node*temp=top;
top=top->next;
delete temp;
return c;}
char peek(){
if(top==NULL)return'\0';
return top->data;}
bool isEmpty(){
return top==NULL;}};
int precedence(char op){
if(op=='^')return 3;
else if(op=='*'||op=='/')return 2;
else if(op=='+'||op=='-')return 1;
else return 0;}
bool isOperand(char c){
return(c>='a'&&c<='z')||
(c>='A'&&c<='Z')||(c>='0'&&c<='9');}
string infixToPostfix(string infix){
Stack st;
string postfix="";
for(int i=0;i<infix.length();i++){
char c=infix[i];
if(isOperand(c)){
postfix+=c;}
else if(c=='('){
st.push(c);}
else if(c==')'){
while(!st.isEmpty()&&st.peek()!='('){
postfix+=st.pop();}
st.pop();}
else{
while(!st.isEmpty()&&
precedence(st.peek())>=precedence(c)){
postfix+=st.pop();}
st.push(c);}}
while(!st.isEmpty()){
postfix+=st.pop();}
return postfix;}
int main(){
string infix;
cout<<"Enter infix expression: ";
cin>>infix;
string postfix=infixToPostfix(infix);
```

```
64    cout<<"Postfix expression: "<<postfix<<endl;
65    return 0;}
```

OUTPUT



Task2:

Evaluate Infix ,prefix and postfix

C: > Users > Nextsol Pvt Ltd > Desktop > C++

```cpp
#include<iostream>
#include<string>
#include<cctype>
using namespace std;
struct Node{
char data;
Node* next;};
class Stack{
Node* top;
public:
Stack(){top=NULL;}
void push(char c){
Node* newNode=new Node();
newNode->data=c;
newNode->next=top;
top=newNode;}
char pop(){
if(top==NULL)return '\0';
char c=top->data;
Node* temp=top;
top=top->next;
delete temp;
return c;}
char peek(){
if(top==NULL)return '\0';
return top->data;}
bool isEmpty(){
return top==NULL;}};
class IntStack{
struct IntNode{
int data;
IntNode* next;};
IntNode* top;
public:
IntStack(){top=NULL;}
void push(int val){
IntNode* newNode=new IntNode();
newNode->data=val;
newNode->next=top;
top=newNode;}
int pop(){
if(top==NULL)return 0;
int val=top->data;
IntNode* temp=top;
top=top->next;
delete temp;
return val;}
bool isEmpty(){
return top==NULL;}};
int precedence(char op){
if(op=='+'||op=='-')return 1;
if(op=='*'||op=='/')return 2;
return 0;}
int applyOp(int a,int b,char op){
if(op=='+')return a+b;
if(op=='-')return a-b;
if(op=='*')return a*b;
if(op=='/')return a/b;
return 0;}
string infixToPostfix(string infix){
```

```
string infixToPostfix(string infix){
Stack s;
string postfix="";
for(int i=0;i<infix.length();i++){
char c=infix[i];
if(isdigit(c)){
postfix+=c;
}else if(c=='('){
s.push(c);
}else if(c==')'){
while(!s.isEmpty()&&s.peek()!='('){
postfix+=s.pop();}
s.pop();
}else{
while(!s.isEmpty()&&precedence(s.peek())>=precedence(c)){
postfix+=s.pop();}
s.push(c);}}
while(!s.isEmpty()){
postfix+=s.pop();}
return postfix;}
int evaluatePostfix(string postfix){
IntStack s;
for(int i=0;i<postfix.length();i++){
char c=postfix[i];
if(isdigit(c)){3
s.push(c-'0');
}else{
int b=s.pop();
int a=s.pop();
int result=applyOp(a,b,c);
s.push(result);}}
return s.pop();}
```

```
int evaluatePrefix(string prefix){
IntStack s;
for(int i=prefix.length()-1;i>=0;i--){
char c=prefix[i];
if(isdigit(c)){
s.push(c-'0');
}else{
int a=s.pop();
int b=s.pop();
int result=applyOp(a,b,c);
s.push(result);}}
return s.pop();}
int evaluateInfix(string infix){
string postfix=infixToPostfix(infix);
return evaluatePostfix(postfix);}
int main(){
int choice;
string expr;
cout<<"Expression Evaluator\n";
cout<<"1. Evaluate Infix\n";
cout<<"2. Evaluate Prefix\n";
cout<<"3. Evaluate Postfix\n";
cout<<"Enter choice: ";
cin>>choice;
cout<<"Enter expression: ";
cin>>expr;
int result;
if(choice==1){
result=evaluateInfix(expr);
cout<<"Infix Result: "<<result<<endl;
}else if(choice==2){
```

```
123    result=evaluatePrefix(expr);
124    cout<<"Prefix Result: "<<result<<endl;
125    }else if(choice==3){
126    result=evaluatePostfix(expr);
127    cout<<"Postfix Result: "<<result<<endl;
128    }else{
129    cout<<"Invalid choice!"<<endl;
130    }return 0;}
```

PREFIX EVALUATE

```
1. Evaluate Infix
2. Evaluate Prefix
3. Evaluate Postfix
Enter choice: 2
Enter expression: 246*+34^
Prefix Result: 2
PS C:\Users\Nextsol Pvt Ltd>
```

OUTPUT

INFIX EVALUATE

```
1. Evaluate Infix
2. Evaluate Prefix
3. Evaluate Postfix
Enter choice: 1
Enter expression: 2+7*(5^7)-2
Infix Result: -2
PS C:\Users\Nextsol Pvt Ltd>
```

POSTFIX EVALUATE

```
1. Evaluate Infix
2. Evaluate Prefix
3. Evaluate Postfix
Enter choice: 3
Enter expression: -+765*7
Postfix Result: 7
PS C:\Users\Nextsol Pvt Ltd>
```

❖ **Conclusion:**

❖ The program helps to understand how expressions are processed in computers.

❖ Using stacks makes the conversion easy and systematic.

❖ It shows the importance of operator precedence and brackets.

❖ Postfix and prefix forms remove the need for parentheses.

❖ The logic improves problem-solving and data structure skills.

❖ It helps in learning how compilers evaluate mathematical expressions.

❖ Overall, the project gives clear understanding of stack applications in C++.